



attempt. If the file is stored somewhere on the disk there will be a way to access it and the game is over before it begins!

You might be thinking that such a file could simply be encrypted, but then you'd just have another password to save somewhere!

This is where hashes come in. Let us first understand what a hash is.

A hash is comparable to a person's fingerprint. While the actual authority lies with the person himself/ herself, for all practical purposes the fingerprint of the persons is considered enough to establish a unique identity. While it is theoretically not impossible that two people have the same fingerprint, the chances of such a thing happening are rather slim.

Similarly, the hash of any data is a fixed size "fingerprint" of that data. If we have the hash of a piece of data – say a password –, it is not possible to get back the original data. How then will such a hash help in securing passwords, or even exposing them?

A hash function is such that given the same data; the computed hash for that data will always be the same. Therefore, if an application chooses to save a hash of a user's password instead of the password in plain text, the application can check if the password a user enters is correct by checking if its hash matches the stored password hash. A weak analogy to a real world case would be, you cannot get back milk from curd, but you can check to see if a white fluid is milk by seeing if it curdles.

A hash is different from encryption, since encryption by its very nature has to be reversible, which a hash is not. A hash is not reversible, and as such, even if one finds out the hash of a user's password, they will be unable to reverse the hashing process to recover the original password. This is not